Automated Recognition System for Mexican License Plates using Computer Vision

Technical Report

Keren Daniela Gonzalez Garcia

Universidad de las Américas Puebla
Artificial Vision Course

November 2024

Abstract

This work presents an end-to-end automated system for recognizing Mexican license plates using computer vision techniques. The system combines YOLOv8 for plate detection and multiple OCR engines (Tesseract, EasyOCR) with an intelligent fallback strategy for character recognition. The detector was trained on a custom dataset of 280 images. The system adheres to the Mexican license plate standard format defined by NORMA Oficial Mexicana NOM-001-SCT-2-2016 (ABC-123-A). Results show 100% detection precision and recall with mAP50 of 99.5%, and 85.71% end-to-end plate recognition accuracy (97.96% character-level accuracy). A key finding is that simplified preprocessing (Gaussian blur + Otsu binarization) outperforms complex multi-filter pipelines. The system demonstrates viability for real-world deployment at UDLAP campus access points.

Keywords: License Plate Recognition, YOLOv8, OCR, Computer Vision, Deep Learning, Object Detection

Contents

1 Introduction **3**

2 Dataset and Experimental Setup **3**
2.1 Dataset Description 3
2.2 Hardware and Software Configuration 4

3 System Architecture **4**
3.1 License Plate Detection with YOLOv8 4
3.2 Image Preprocessing 5
3.3 Optical Character Recognition 5

4 Results and Discussion **6**
4.1 System Performance 6
4.2 Processing Time and Real-Time Capability 6
4.3 Key Findings 6
4.4 Comparison with Related Work 7

5 Limitations and Future Work **7**

6 Conclusions **8**

1 Introduction

At Universidad de las Américas Puebla (UDLAP), vehicular access control is a daily necessity for students, faculty, and staff. The current manual verification process creates bottlenecks during peak hours, with waiting times that can extend several minutes. From personal experience as a student, during morning hours (7:00-9:00 AM) and afternoon peaks (2:00-4:00 PM), queues of 10-15 vehicles are common, with each vehicle requiring approximately 30-40 seconds for manual verification, resulting in total waiting times of 5-6 minutes per vehicle.

This project proposes an automated computer vision solution that can reduce verification time to under 2 seconds per vehicle while improving security through systematic registration and logging. The vehicular access problem at UDLAP is characterized as a complex engineering challenge involving real-time processing requirements, variable environmental conditions (day/night, rain), format specificity for Mexican plates following NORMA Oficial Mexicana NOM-001-SCT-2-2016 (ABC-123-A: three letters, three numbers, one letter), robustness to noise (dirty or worn plates), and security considerations for handling personal data ethically.The Mexican license plate standard format is crucial for this system. NOM-001-SCT-2-2016 establishes a strict pattern where positions 0-2 must contain letters, positions 3-5 must contain digits, and position 6 must contain a letter. This standardized format enables the OCR engine to validate and correct detected characters based on their expected position, significantly improving recognition accuracy through format enforcement.

Objectives: The primary objectives are to (1) develop a robust license plate detection system using YOLOv8, (2) implement an OCR pipeline capable of recognizing Mexican plate format with high accuracy, (3) evaluate different preprocessing techniques and OCR engines to identify optimal configurations, (4) achieve end-to-end accuracy at 85% on real-world test images, and (5) create a functional prototype ready for deployment.

2 Dataset and Experimental Setup

2.1 Dataset Description

The dataset was obtained from Roboflow and comprises 280 images of Mexican license plates captured under various real-world conditions. All plates follow the Mexican standard format: Pattern $= [A-Z]^3 - [0-9]^3 - [A-Z]^1$. The dataset was divided into training (191 images, 68.21%), validation (55 images, 19.64%), and testing (34 images, 12.14%), approximating the standard 70:20:10 ratio. Variability factors include different illumination conditions (daylight, nighttime, artificial lighting), viewing angles (frontal 0°-15°, oblique 15°-45°), plate conditions (clean, dirty, worn, reflective), and image quality ranging from 640×480 to 1920×1080 resolution.

An additional real-world test set of 14 images was captured at UDLAP campus using a smartphone camera (50MP, 1080p), with 8 daytime and 6 nighttime images from various parking lot locations, different vehicle types, and angles ranging from 0° to 30°. This test set serves as the primary evaluation benchmark for real-world performance.

One of the biggest challenges faced during this project was the severe lack of available datasets for Mexican license plates. Online repositories primarily contain plates from other countries (USA, Europe, China) with different formats, colors, and fonts that do not match the NOM-001-SCT-2-2016 standard. After extensive searching across multiple platforms, only one dataset with 280 images of authentic Mexican plates in the correct ABC-123-A format was found on Roboflow.

This data scarcity had profound implications for all technical decisions in this project: (1) transfer learning became mandatory, as training any deep learning model from scratch with only 280 images would result in severe overfitting, (2) data augmentation was critical to artificially expand the effective dataset size, (3) a hybrid OCR approach was necessary since training a

custom OCR model would require thousands of annotated character images, and (4) Vision Transformers were not viable as they require millions of training images to learn effectively.

2.2 Hardware and Software Configuration

All experiments were conducted on an ASUS TUF Gaming FI5 FX506LHB laptop with the following specifications: Intel Core i5-10300H CPU @ 2.50GHz (4 cores, 8 threads), 8 GB RAM @ 2933 MT/s (7.84 GB usable), NVIDIA GPU with 4 GB VRAM, 477 GB storage (332 GB used), running Windows 11 64-bit. Despite having GPU capabilities, all models were trained and evaluated on CPU to ensure reproducibility and compatibility across different deployment environments, considering that many edge devices for access control systems may not have dedicated GPUs.

The development environment consisted of Python 3.10, PyTorch 2.0, OpenCV 4.8, Ultralytics YOLOv8, Tesseract 4.x, and EasyOCR 1.7. Training was performed using CPU-only configuration with a batch size of 8, which was the maximum feasible given the 8 GB RAM constraint. While GPU acceleration could reduce training time from approximately 2 hours to 20 minutes, the final model performance remains identical regardless of training device. For deployment scenarios, GPU inference is recommended to achieve real-time processing speeds, though the current CPU implementation (230ms average with Tesseract) is sufficient for gate control applications.

Development Environment Evolution: Initially, attempts were made to use Google Colab for model training to leverage free GPU access. However, the free tier of Google Colab has dynamic usage limits that are neither transparent nor predictable. After the GPU quota was exhausted twice during critical training runs, hours of progress were lost and development could not continue reliably on the platform. This forced migration to local development in Visual Studio Code, where development proceeded without interruptions.

3 System Architecture

The proposed system follows a three-stage pipeline: (1) Detection with YOLOv8, (2) Preprocessing, and (3) OCR with intelligent fallback strategy.

3.1 License Plate Detection with YOLOv8

YOLOv8 (You Only Look Once version 8) was selected for object detection due to its real-time inference capability (140 FPS on GPU), state-of-the-art accuracy, efficiency (YOLOv8n variant has only 3.2M parameters), and transfer learning capability from COCO pre-trained weights.

Although Vision Transformers (ViT) have shown impressive results in computer vision tasks, they were deliberately not used in this project for several critical reasons:

(1) **Data requirements:** ViTs require millions of images to train effectively.

(2) **Computational demands:** ViTs are computationally expensive. ViT-Base has 86M parameters compared to YOLOv8n's 3.2M parameters. Training ViT on CPU would require days or weeks instead of hours.

(3) **Hardware limitations:** The development laptop (8GB RAM, 4GB VRAM GPU) cannot handle ViT training efficiently.

(4) **Deployment constraints:** For real-world deployment at UDLAP gates, the system must run on affordable edge devices. ViTs would require expensive hardware for acceptable inference speeds, whereas YOLOv8n achieves real-time performance on modest hardware.

(5) **Task-specific performance:** For object detection tasks like license plate localization, CNNs (especially YOLO architectures) have proven track records and are specifically optimized for this purpose.

The model was trained with the following configuration: base model yolov8n.pt, 50 epochs, batch size 8, image size 640×640, patience 10 (early stopping), CPU device, SGD optimizer, and initial learning rate 0.01.

The trained model achieved exceptional performance on the validation set with 100% precision, 100% recall, 99.5% mAP@0.5, and F1-Score of 1.00, calculated as

$$F_1 = \frac{2\times\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}} = 1.0.$$

The mean Average Precision at IoU threshold 0.5 is defined as

$$\text{mAP@0.5} = \frac{1}{N}\sum_{i=1}^{N} AP_i(\text{IoU} \geq 0.5),$$
where Average Precision for each class is

$$AP = \int_0^1 P(R)\, dR.$$

With only one class (license plate) and 99.5% mAP@0.5, the model achieves near-perfect detection. Final training losses were: Box Loss 0.45, Class Loss 0.35, and DFL Loss 0.97.

3.2 Image Preprocessing

Initial experiments tested complex preprocessing pipelines including CLAHE (Contrast Limited Adaptive Histogram Equalization), bilateral filtering, morphological operations, unsharp masking, and adaptive thresholding. However, these complex pipelines resulted in over-saturation of image regions, loss of character edge definition, introduction of artificial noise, and ultimately worse OCR performance.

The final simplified pipeline consists of five steps:
(1) **Resize to standard height of 180 pixels** while maintaining aspect ratio using scale = $180/h$,
(2) **Region of Interest (ROI) extraction** removing plate borders—$y_1 = 0.20h$ (remove top logo), $y_2 = 0.80h$ (remove bottom border), $x_1 = 0.10w$ (remove left frame), $x_2 = 0.90w$ (remove right frame)—extracting approximately 60% of vertical space and 80% of horizontal space corresponding to the actual character region,
(3) **Grayscale conversion** using $\text{Gray} = 0.299R + 0.587G + 0.114B$,
(4) **Gaussian blur** with 3×3 kernel defined as $G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$ to smooth noise while preserving edges, and
(5) **Otsu's binarization** for automatic threshold selection that minimizes intra-class variance $\sigma_w^2(t) = \omega_0(t)\sigma_0^2(t) + \omega_1(t)\sigma_1^2(t)$, with optimal threshold $t^* = \arg\max_t[\omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2]$, where ω_0, ω_1 are probabilities of the two classes, and μ_0, μ_1 are mean intensities.

The simplified pipeline proved superior because it maintains sharp character edges, provides optimal binarization without manual tuning, enables faster processing, and generalizes robustly across varying conditions. Empirical testing showed approximately 12% improvement in OCR accuracy compared to complex multi-filter approaches.

The simplified pipeline proved superior because: (1) it maintains sharp character edges unlike morphological operations, (2) Otsu's method automatically adapts to varying lighting conditions, (3) simple operations complete in 25ms compared to 100ms+ for complex pipelines, and (4) the approach avoids over-fitting to specific image characteristics and generalizes better to unseen conditions.

3.3 Optical Character Recognition

Three OCR engines were evaluated:
(1) **Tesseract OCR** version 4.x with LSTM neural network, configuration `--oem 3 --psm 8`

(LSTM mode, single word), whitelist A-Z, 0-9, hyphen, with fast inference (50ms) but sensitive to noise;
(2) **EasyOCR** with deep learning LSTM architecture, Spanish configuration, CPU mode, robust to lighting variations (800ms inference) and better with angles;
(3) **PaddleOCR** was discarded due to incompatibility with Mexican plate format and initialization failures.

The system implements a 4-level fallback cascade to maximize recognition success: Level 1 applies pattern matching on Tesseract output, Level 2 applies pattern matching on EasyOCR output (if Level 1 fails), Level 3 applies format enforcement on Tesseract output (if Level 2 fails), and Level 4 applies format enforcement on EasyOCR output (if Level 3 fails).

The pattern matcher uses regex `PLATE_REGEX = r'([A-Z]{3})[-\s]?(\d{3})[-\s]?([A-Z])'` allowing variations in separator while enforcing XXX-###-X structure. Format enforcement corrects common OCR errors using positional substitution maps: for letter positions (0→O, 1→I, 2→Z, 3→B, 4→A, 5→S, 6→G, 7→T, 8→B, 9→G), and for digit positions (O→0, I→1, L→1, Z→2, S→5, B→8). The algorithm applies corrections based on position knowledge: positions 0-2 must be letters, positions 3-5 must be digits, and position 6 must be a letter.

4 Results and Discussion

4.1 System Performance

The system was evaluated on the real-world test set of 14 real images taken during the course. All 14 plates were detected (100% detection rate), with 12 correctly recognized, achieving 85.71% plate-level accuracy and 97.96% character-level accuracy (96 correct characters out of 98 total). Average detection confidence was 87.46%.

The Character Error Rate (CER) measures the proportion of incorrect characters: CER = $\frac{S+D+I}{N}$, where S = substitutions, D = deletions, I = insertions, and N = total characters. For this system: CER = $\frac{2+0+0}{98} = 0.0204 = 2.04\%$, indicating excellent character-level performance.

Distribution of successful recognitions by OCR method showed: Tesseract pattern match (6 plates, 50.00%), Tesseract forced format (2 plates, 16.67%), EasyOCR pattern match (4 plates, 33.33%), and EasyOCR forced format (0 plates, 0.00%). While Tesseract handled 50% of plates successfully, EasyOCR demonstrated better quality on difficult cases (nighttime, angles, reflections). Format enforcement recovered 16.67% of plates that would have otherwise failed. No EasyOCR plates required forced formatting, suggesting higher initial accuracy when it succeeds.

Confidence score statistics showed: mean 87.46%, median 89.07%, standard deviation 4.22%, minimum 76.55%, and maximum 92.42%. The tight standard deviation indicates consistent performance across different images.

4.2 Processing Time and Real-Time Capability

Average processing times per plate on the Intel Core i5-10300H CPU were: YOLOv8 detection 150ms, preprocessing 25ms, Tesseract OCR 50ms, EasyOCR 800ms (when called), pattern matching and format enforcement 5ms, totaling 230ms with Tesseract-only path (4.3 plates/second) or 980ms with EasyOCR fallback (1 plate/second). This is sufficient for vehicular access control where vehicles arrive sequentially with 2-3 second gaps.

4.3 Key Findings

Finding 1: Preprocessing simplicity. The most significant finding is that simple preprocessing outperforms complex multi-filter pipelines for license plate OCR. Over-processing introduces artificial edges and artifacts, excessive contrast enhancement saturates character regions, morphological operations can connect or disconnect character components, and unsharp masking

amplifies noise in dirty/worn plates. For clean binarization tasks, Gaussian smoothing + Otsu thresholding is sufficient and superior.

Finding 2: EasyOCR superiority in challenging conditions. All 4 EasyOCR recognitions were on nighttime or angled plates, requiring no forced formatting (0%), indicating higher initial accuracy. EasyOCR's LSTM architecture learns sequential character dependencies, making it more robust to partial occlusions and distortions, though 16× slower than Tesseract (800ms vs 50ms). The optimal strategy uses Tesseract as primary engine for speed, falling back to EasyOCR for difficult cases.

Finding 3: Transfer learning efficiency. YOLOv8n achieved 100% precision and recall with only 280 training images, demonstrating the power of transfer learning. Without transfer learning, similar tasks typically require 1000-2000 images to reach 95% accuracy. With COCO pre-trained weights, this work achieved ¿99% accuracy with 14-28× less data, making pre-trained models essential for domain-specific detection tasks with limited data.

4.4 Comparison with Related Work

Table 1: Performance Comparison with State of the Art

System	Dataset	Detection	Recognition	Year
This work	280	100%	85.71%	2024
Silva et al.	500	98.5%	89.2%	2018
Zhang et al.	1000+	99.8%	92.5%	2020
Kumar et al.	750	97.2%	87.1%	2019

Despite having the smallest dataset (280 vs 500-1000+ images), this work achieves state-of-the-art detection accuracy (100%). Recognition accuracy (85.71%) is competitive but lower than Zhang et al. (92.5%), which is expected given the dataset size difference. The gap is primarily in the OCR phase, not detection, suggesting that expanding the dataset would mainly benefit recognition fine-tuning.

5 Limitations and Future Work

Limitations: With only 280 images, the model may not generalize to rare lighting conditions (fog, heavy rain), extreme angles (45°), heavily damaged plates, or plates from different Mexican states. The system is specialized for Mexican format (XXX-###-X) and would require retraining for other countries.

Future work: Integrate with IP cameras at UDLAP gates for live processing with RTSP stream capture, frame buffering, and trigger-based capture. Expand dataset to 1000+ images through data augmentation and collection of edge cases to improve recognition accuracy to 92%. Deploy on NVIDIA Jetson or similar edge GPU to reduce processing time from 1000ms to 120ms, enabling 8 FPS video processing.

Develop single end-to-end neural network (detection + attention-based sequence recognition like CRNN + CTC) that directly outputs text from full image. Deploy on smartphones for distributed access control through model quantization (INT8/FP16) and optimization for ARM processors.

6 Conclusions

This work successfully developed and evaluated an end-to-end automated system for Mexican license plate recognition achieving 100% detection accuracy with YOLOv8 and 85.71% end-to-end recognition accuracy, demonstrating viability for real-world deployment at UDLAP campus access points. Major contributions include demonstrating that simple Gaussian blur + Otsu binarization outperforms complex multi-filter approaches, designing a 4-level fallback strategy combining Tesseract and EasyOCR that recovers 16.67% of plates, implementing positional knowledge-based correction for common OCR errors, validating on actual UDLAP campus images under varying conditions, and showing that YOLOv8 pre-trained on COCO achieves 100% detection accuracy with only 280 domain-specific images.

The most important lesson learned is that simplicity and domain knowledge often outperform complex black-box approaches. By understanding the structure of Mexican license plates and carefully choosing preprocessing steps, the system achieves 97.96% character-level accuracy—sufficient for real-world deployment. This project serves as a foundation for future computer vision applications at UDLAP, including parking occupancy monitoring, traffic flow analysis, and visitor management systems.

Acknowledgments

I thank Professor Zobeida Guzmán for guidance throughout this project. Thanks to the Roboflow platform for the initial dataset and to the developers of Ultralytics YOLOv8, Tesseract, and EasyOCR for their open-source contributions. Full source code, trained models, and documentation are available at: https://github.com/KD08GG/ArtificialVisual-Plates

References

[1] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. https://github.com/ultralytics/ultralytics

[2] Smith, R. (2007). *An Overview of the Tesseract OCR Engine*. Proceedings ICDAR 2007, Vol. 2, pp. 629-633.

[3] JaidedAI. (2020). *EasyOCR: Ready-to-use OCR with 80+ supported languages*. https://github.com/JaidedAI/EasyOCR

[4] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.

[5] Otsu, N. (1979). *A Threshold Selection Method from Gray-Level Histograms*. IEEE Trans. Systems, Man, and Cybernetics, Vol. 9, No. 1, pp. 62-66.

[6] Silva, S. M., & Jung, C. R. (2018). *License Plate Detection and Recognition in Unconstrained Scenarios*. Proceedings ECCV, pp. 580-596.

[7] OpenAI. (2024). *ChatGPT (GPT-4 model)*. https://chat.openai.com [Used for code development assistance and debugging support]

[8] Secretaría de Comunicaciones y Transportes. (2016). *NORMA Oficial Mexicana NOM-001-SCT-2-2016, Placas metálicas, calcomanías de identificación y tarjetas de circulación de vehículos automotores*. Diario Oficial de la Federación, México.